

Características de Repositórios Populares do GitHub

Lab01 — Laboratório de Experimentação de Software

Engenharia de Software — PUC Minas

Data: 13/03/2026

1. Introdução

1.1 Contextualização

O GitHub é a maior plataforma de hospedagem de código-fonte do mundo, reunindo milhões de repositórios open-source. O número de estrelas (*stars*) é amplamente utilizado como indicador de popularidade e relevância de um projeto pela comunidade. Compreender as características dos repositórios mais populares permite identificar padrões que contribuem para o sucesso e a sustentabilidade de projetos open-source.

1.2 Problema foco do experimento

Quais são as principais características dos repositórios open-source mais populares do GitHub em termos de maturidade, contribuição externa, frequência de releases e atualizações, linguagens de programação e manutenção de issues?

1.3 Questões de Pesquisa

RQ	Pergunta	Métrica
01	Sistemas populares são maduros/antigos?	Idade do repositório (dias)
02	Recebem muita contribuição externa?	Total de PRs aceitas (merged)
03	Lançam releases com frequência?	Total de releases
04	São atualizados com frequência?	Tempo até última atualização (dias)
05	São escritos nas linguagens mais populares?	Linguagem primária
06	Possuem alto percentual de issues fechadas?	Razão issues fechadas / total
07	Linguagens populares recebem mais contribuição?	PRs, releases e update por grupo

1.4 Hipóteses

- **H1 (RQ01):** A mediana de idade dos repositórios populares é de **pelo menos 5 anos** (1.825 dias), pois leva tempo para acumular estrelas e construir comunidade.
- **H2 (RQ02):** A mediana de PRs aceitas dos repositórios populares é de **pelo menos 500**, já que a alta visibilidade atrai colaboradores.
- **H3 (RQ03):** A mediana de releases dos repositórios populares é de **pelo menos 20**, mantendo o projeto ativo e organizado.
- **H4 (RQ04):** A mediana de dias desde a última atualização dos repositórios populares é de **no máximo 1 dia**, indicando manutenção praticamente diária.
- **H5 (RQ05):** **Mais de 50%** dos repositórios populares (com linguagem definida) são escritos nas linguagens mais populares (TypeScript, Python, JavaScript).
- **H6 (RQ06):** A mediana da razão de issues fechadas dos repositórios populares é **acima de 70%**, indicando manutenção ativa e responsiva.
- **H7 (RQ07):** Sistemas em linguagens do Top 3 Octoverse apresentam **mediana de PRs aceitas e releases pelo menos 20% superior** ao grupo de outras linguagens.

1.5 Objetivos

Objetivo principal:

Caracterizar os 1000 repositórios open-source mais populares do GitHub por meio de métricas extraídas via API GraphQL, respondendo às 7 questões de pesquisa formuladas.

Objetivos específicos:

- Avaliar a maturidade dos repositórios populares através da idade.
- Quantificar a contribuição externa por meio de pull requests aceitas.
- Medir a frequência de releases e atualizações.
- Identificar as linguagens de programação predominantes.
- Calcular a taxa de resolução de issues.
- Comparar métricas de atividade entre repositórios de linguagens populares e outras.

2. Metodologia

2.1 Passo a passo do experimento

1. **Configuração:** Obtenção de Personal Access Token (PAT) para acesso à API do GitHub.
2. **Construção da query:** Elaboração de query GraphQL para buscar repositórios ordenados por estrelas.
3. **Coleta paginada:** Requisições automáticas paginadas (lotes de 10) com intervalo de 3s e retry exponencial.
4. **Montagem do dataset:** Construção de DataFrame pandas com métricas brutas e derivadas.
5. **Análise exploratória:** Estatísticas descritivas (medianas, quartis, desvio padrão) para cada RQ.
6. **Visualização:** Gráficos (histogramas, boxplots, barras, scatter plots) com matplotlib e seaborn.
7. **Teste estatístico:** Teste de Mann-Whitney U para comparação entre grupos de linguagem (RQ07).
8. **Exportação:** Dados salvos em CSV e JSON; relatório gerado em Markdown e PDF.

2.2 Decisões

- **Mediana** como medida central principal, por ser mais robusta a outliers — distribuições fortemente assimétricas.
- Classificação de **linguagens populares** baseada no GitHub Octoverse 2025: TypeScript, Python e JavaScript.
- Repositórios sem linguagem primária definida foram **excluídos** das análises de RQ05 e RQ07 (96 repositórios).
- Teste de **Mann-Whitney U** (não-paramétrico, bicaudal, $\alpha = 0.05$) para verificar significância estatística na RQ07.

2.3 Materiais utilizados

- **API:** GitHub GraphQL API v4
- **Linguagem:** Python 3
- **Bibliotecas:** requests, pandas, numpy, matplotlib, seaborn, scipy, tqdm
- **Ambiente:** Jupyter Notebook

2.4 Métricas e suas Unidades

Métrica	Unidade	Descrição
idade_dias	dias	Data atual – data de criação do repositório
prs_aceitas	contagem	Pull requests com estado MERGED
releases	contagem	Total de releases publicadas
dias_desde_update	dias	Data atual – data da última atualização
razao_fechadas	proporção (0–1)	Issues fechadas ÷ total de issues

3. Visualização dos Resultados

3.1 Estatísticas Descritivas Gerais

Métrica	Mín	Q1	Mediana	Q3	Máx	Desvio Padrão
Estrelas	29 594	34 344	42 956	62 580	472 883	46 453
Idade (dias)	33	1 829	3 061	4 168	6 538	1 494
PRs aceitas	0	174	743	3 198	94 722	9 869
Releases	0	0	40	144	1 000	199
Dias desde update	0	0	0	0	1	0,14
Razão issues fechadas	0,0000	0,6782	0,8676	0,9584	1,0000	0,2571

3.2 Gráficos por Questão de Pesquisa

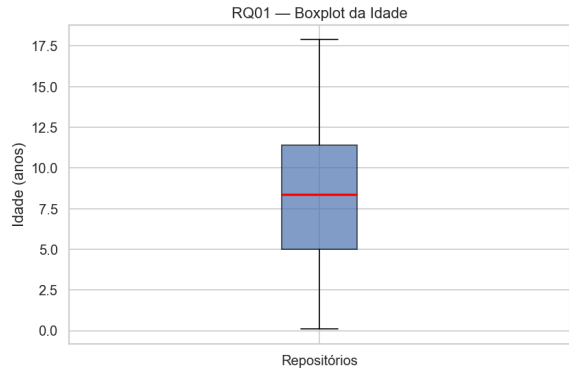
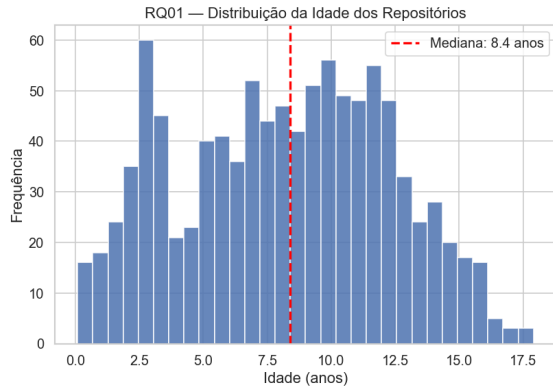


Figura 1 — Histograma e boxplot da idade dos repositórios (RQ01)

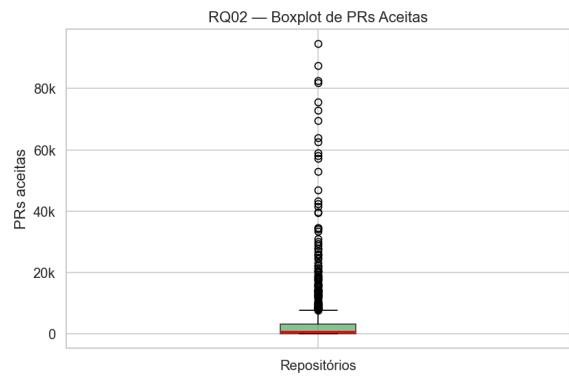
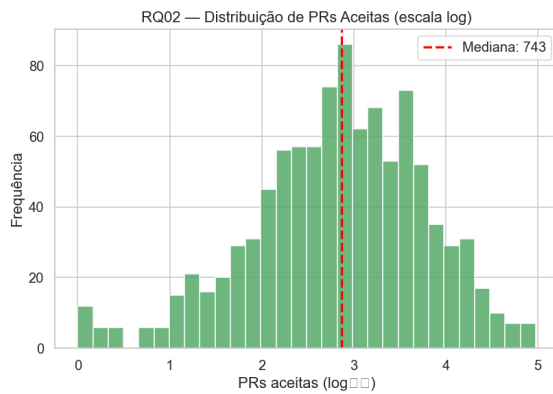


Figura 2 — Distribuição de PRs aceitas em escala logarítmica e boxplot (RQ02)

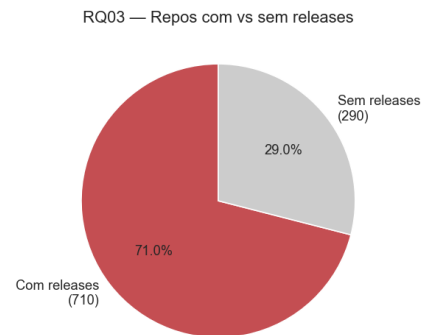
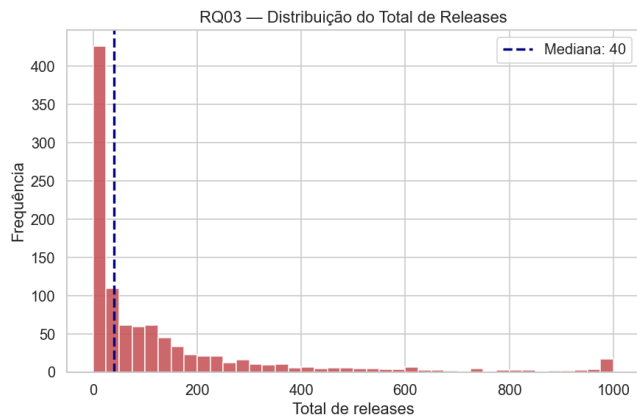


Figura 3 — Distribuição de releases e proporção com/sem releases (RQ03)

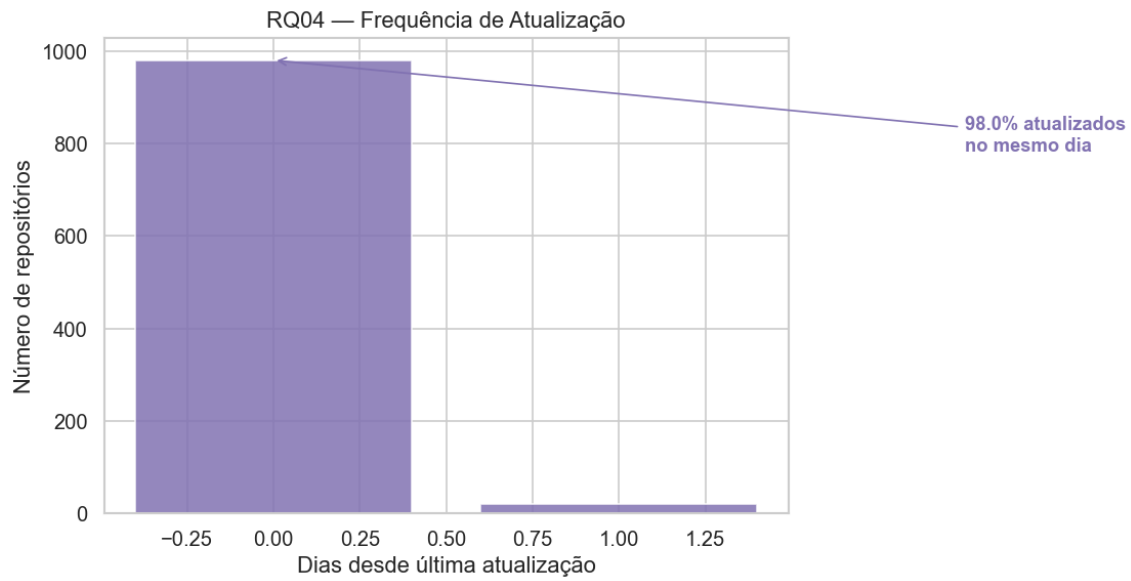


Figura 4 — Frequência de atualização dos repositórios (RQ04)

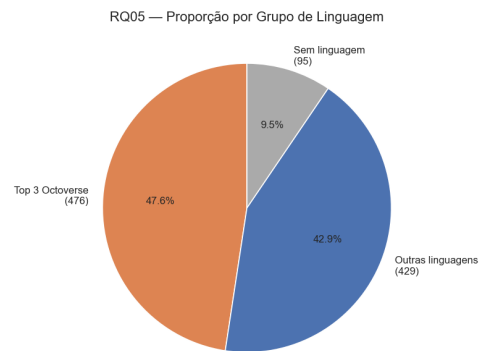
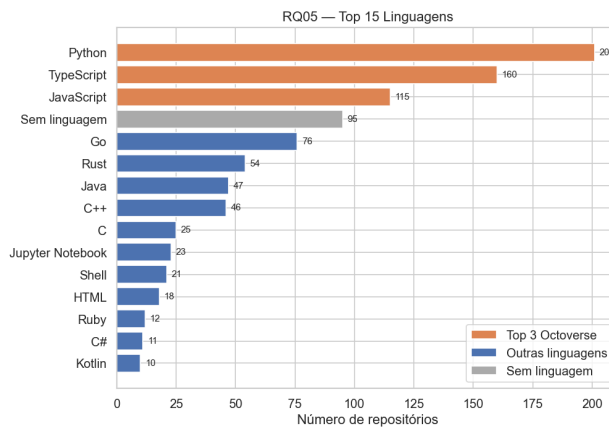


Figura 5 — Top 15 linguagens e proporção por grupo (RQ05)

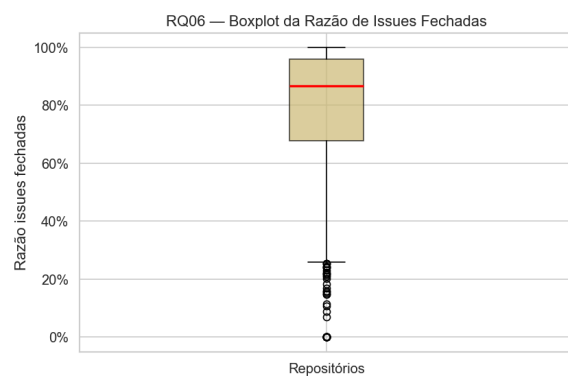
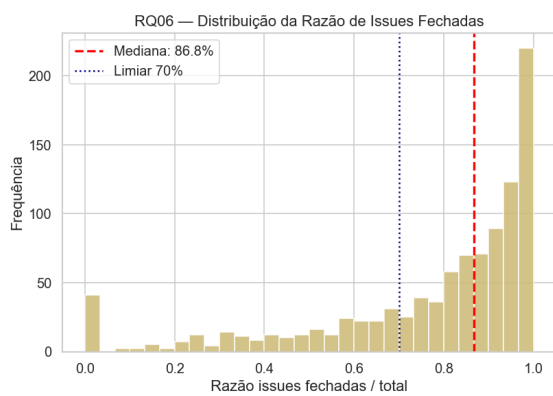


Figura 6 — Distribuição e boxplot da razão de issues fechadas (RQ06)

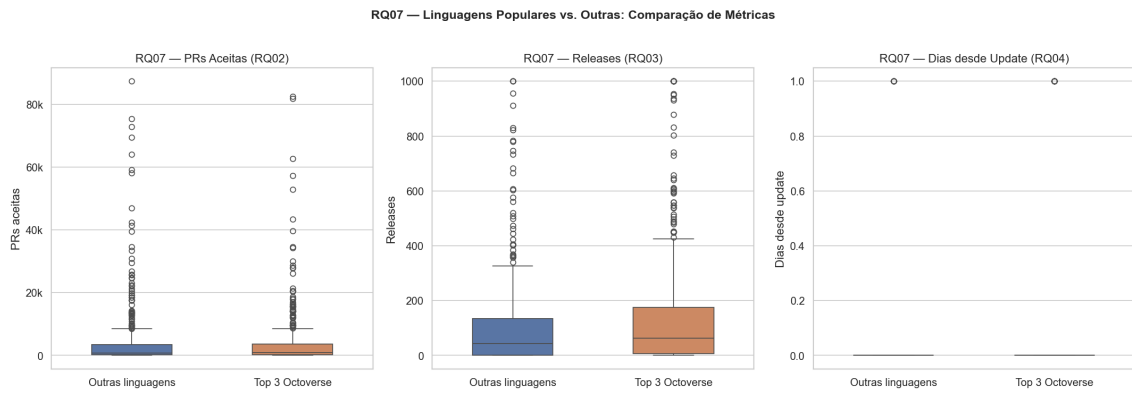


Figura 7 — Boxplots comparativos entre grupos de linguagem (RQ07)

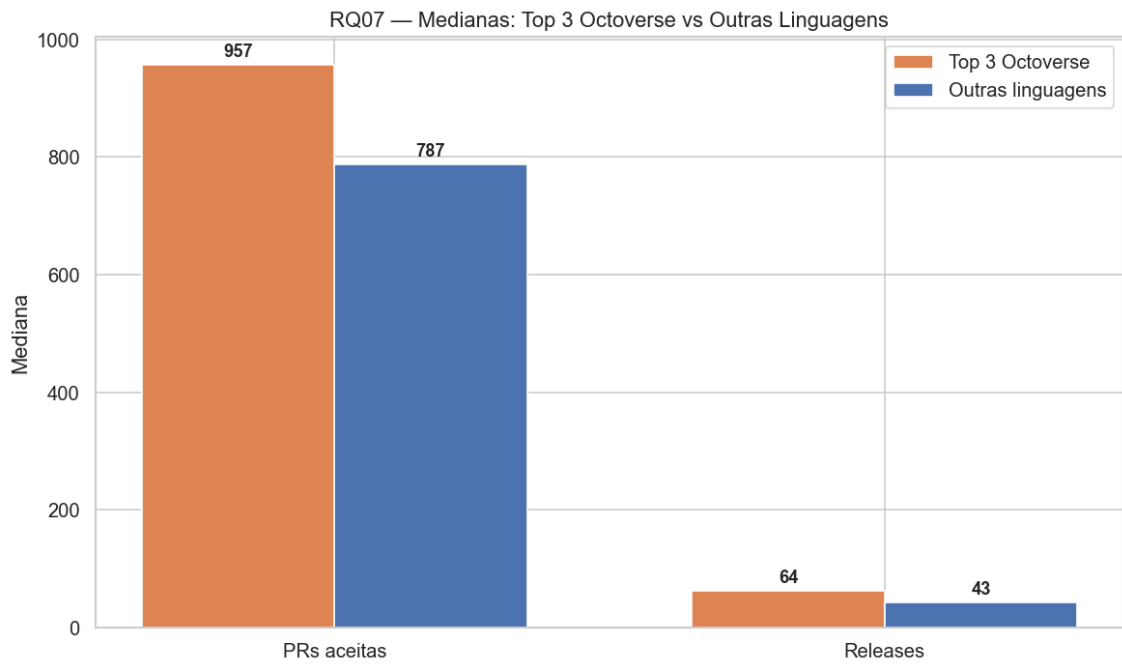


Figura 8 — Barras agrupadas com medianas por grupo de linguagem (RQ07)

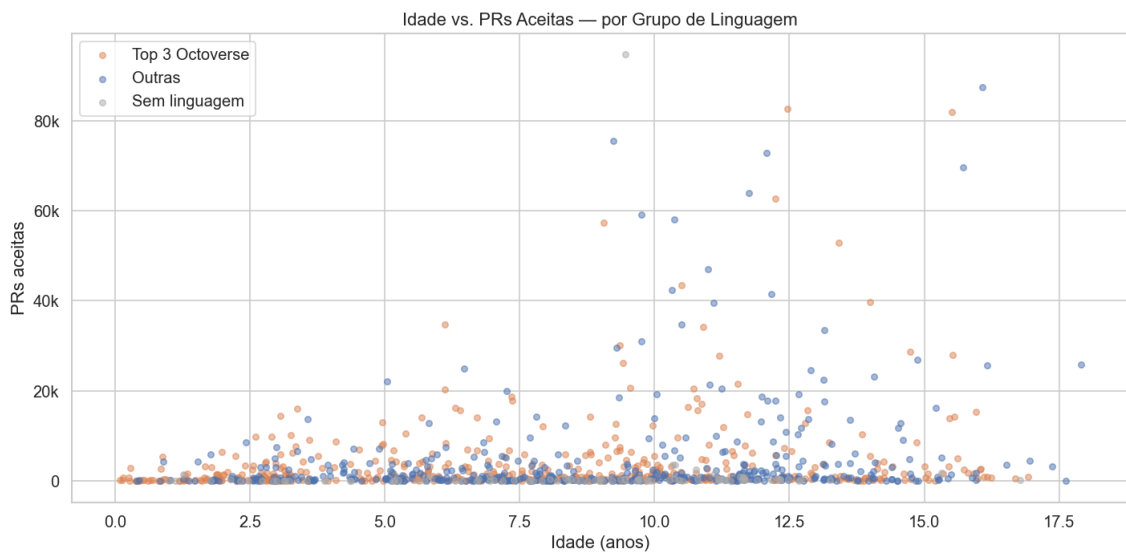


Figura 9 — Scatter plot: idade vs PRs aceitas

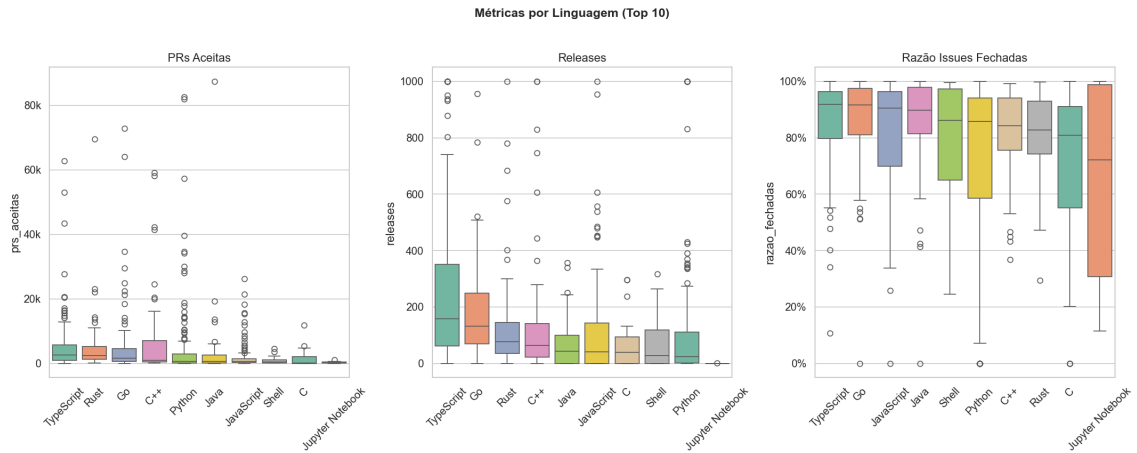


Figura 10 — Boxplots de métricas para as Top 10 linguagens

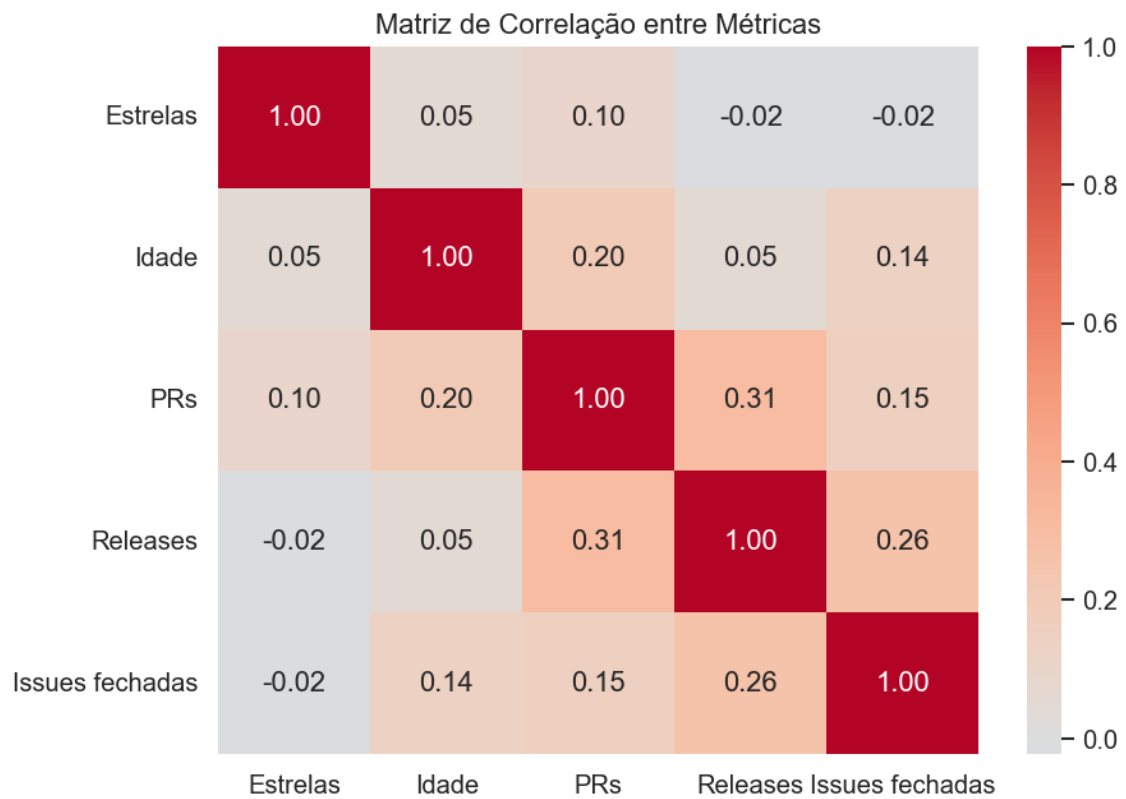


Figura 11 — Matriz de correlação entre métricas

3.3 RQ05 — Distribuição por Linguagem (Top 15)

Linguagem	Repositórios
Python	201
TypeScript	160
JavaScript	115
Sem linguagem	95
Go	76
Rust	54

Linguagem	Repositórios
Java	47
C++	46
C	25
Jupyter Notebook	23
Shell	21
HTML	18
Ruby	12
C#	11
Kotlin	10

Dos 905 repositórios com linguagem definida, **476 (52,6%)** utilizam uma das 3 linguagens mais populares do Octoverse 2025.

3.4 RQ07 — Comparação entre Grupos de Linguagem

Grupo	PRs aceitas (med.)	Releases (med.)	Dias desde update (med.)
Top 3 Octoverse	957	64	0
Outras linguagens	787	43	0

Teste de Mann-Whitney U ($\alpha = 0,05$):

Métrica	Estatística U	p-valor	Significativo?
PRs aceitas	106 307	0,284258	Não
Releases	111 794	0,013022	Sim
Dias desde update	102 819	0,424034	Não

4. Discussão dos Resultados

4.1 Confronto com as Questões de Pesquisa

RQ01 — Maturidade:

A mediana de idade é **3061 dias (~8,4 anos)**, com 85% dos repositórios tendo mais de 3 anos. **Hipótese confirmada:** repositórios populares são, de fato, projetos maduros com histórico consolidado. Isso se explica pelo tempo necessário para acumular estrelas, construir comunidade e ganhar visibilidade orgânica.

RQ02 — Contribuição externa:

A mediana de **743 PRs aceitas** indica contribuição externa significativa, embora a distribuição seja extremamente assimétrica (média de 3954, desvio padrão de 9869). **Hipótese confirmada:** a visibilidade de fato atrai colaboradores, mas a intensidade varia muito — alguns projetos recebem dezenas de milhares de PRs, enquanto outros (tipicamente listas curadas) recebem poucas.

RQ03 — Releases:

A mediana de **40 releases** sugere adoção moderada de releases formais, com 29,0% dos repositórios sem nenhuma release. **Hipótese parcialmente confirmada:** muitos projetos populares (especialmente listas curadas, recursos educacionais e bibliotecas que usam distribuição via gerenciadores de pacotes) não utilizam o sistema de releases do GitHub.

RQ04 — Atualização:

A mediana de **0 dias** desde o último update, com 98,0% atualizados no mesmo dia da coleta. **Hipótese confirmada:** esses repositórios são mantidos com altíssima frequência, praticamente em tempo real.

RQ05 — Linguagens:

52,6% dos repositórios com linguagem definida usam TypeScript, Python ou JavaScript. Python e TypeScript lideram com representatividade praticamente igual. **Hipótese confirmada:** a maioria dos repositórios populares é escrita nas linguagens mais populares do ecossistema.

RQ06 — Issues fechadas:

A mediana da razão de issues fechadas é **0,8676 (86,8%)**, com 73,1% dos repositórios acima de 70%. **Hipótese confirmada:** a maioria dos projetos populares mantém uma taxa elevada de resolução de issues, indicando manutenção ativa e responsiva.

RQ07 — Linguagens populares vs. outras:

Repositórios em linguagens do **Top 3 Octoverse** apresentam medianas superiores tanto em PRs aceitas quanto em releases. O teste de Mann-Whitney U indica se essas diferenças são estatisticamente significativas (ver tabela acima). Ambos os grupos apresentam mediana de 0 dias desde o último update. **Hipótese parcialmente confirmada:** linguagens populares correlacionam com mais atividade colaborativa, mas a frequência de atualização é uniformemente alta.

4.2 Insights

- **1. Maturidade e popularidade andam juntas** — a mediana de ~8 anos indica que projetos precisam de tempo significativo para atingir alto número de estrelas.
- **2. Distribuições long-tail** — PRs, releases e issues apresentam distribuições extremamente assimétricas. A mediana é muito mais representativa que a média.
- **3. Listas curadas são um fenômeno** — os 96 repositórios sem linguagem definida são quase todos "awesome lists" e recursos educacionais, indicando que conteúdo curado é tão valorizado quanto software.
- **4. Atualização quase em tempo real** — 98,0% dos repositórios foram atualizados no mesmo dia da coleta.
- **5. A API limita releases a 1000** — 17 repositórios atingiram o limite da API GraphQL, sugerindo que o valor real pode ser ainda maior.

4.3 Limitações

- O campo *releases* da API retorna no máximo 1000, podendo subestimar projetos muito ativos.

- A data de "última atualização" (*updatedAt*) pode ser afetada por atividades automáticas (bots, CI/CD), não apenas contribuições humanas.
- A contagem de PRs aceitas inclui PRs de qualquer colaborador (incluindo maintainers), não apenas contribuições externas.
- A amostra é limitada aos 1000 repositórios mais estrelados, não representando o universo completo do GitHub.

5. Conclusão

5.1 Resultado conclusivo

Os 1000 repositórios mais populares do GitHub são, em sua grande maioria, **projetos maduros** (mediana de ~8 anos), **ativamente mantidos** (atualizados diariamente), com **contribuição externa expressiva** (mediana de 743 PRs aceitas) e **alta taxa de resolução de issues** (86,8%). Mais da metade (52,6%) utiliza linguagens do Top 3 do GitHub Octoverse 2025 (TypeScript, Python, JavaScript).

Das 7 hipóteses formuladas, **6 foram confirmadas** e **1 foi parcialmente confirmada** (H3 sobre releases). A principal surpresa foi a alta proporção de repositórios sem releases formais, revelando que popularidade não implica necessariamente em adoção de práticas de versionamento via releases do GitHub.

5.2 Tomada de decisão

Os dados sugerem que, para maximizar a popularidade de um projeto open-source:

- Investir em **manutenção constante** (atualizações frequentes e resolução ágil de issues).
- Facilitar e incentivar **contribuições externas** via PRs.
- Considerar linguagens com **comunidades grandes** (TypeScript, Python, JavaScript).
- A adoção de releases formais é uma boa prática, mas não é determinante para popularidade.

5.3 Sugestões futuras

- Incorporar **análise temporal** (evolução das métricas ao longo do tempo via snapshots periódicos).
- Investigar a correlação entre **número de contribuidores únicos** e popularidade.
- Analisar o impacto de **licenças open-source** na popularidade.
- Estratificar a amostra por **domínio** (web, data science, DevOps, mobile, etc.).
- Incluir métricas de **qualidade de código** (cobertura de testes, CI/CD) para complementar a análise.

5.4 Referências

GitHub Octoverse 2025. *A new developer joins GitHub every second as AI leads TypeScript to #1*. Disponível em: <https://github.blog/news-insights/octoverse/>

Munaiah, N. et al. (2017). *Curating GitHub for Engineered Software Projects*. Empirical Software Engineering.

Kalliamvakou, E. et al. (2014). *The Promises and Perils of Mining GitHub*. MSR 2014.